

Lesson 11 — Polynomial Addition Algorithm

1. Where We Are in the Course

You have already completed the sparse matrix part of the course and started the polynomial part.

Before Lesson 11, you learned:

Lesson	Main idea
Lesson 1	Why sparse matrices save memory
Lesson 2	Coordinate List representation and creation
Lesson 3	Complete C sparse matrix display program
Lesson 4	Sparse matrix addition algorithm
Lesson 5	Program to add two sparse matrices
Lesson 6	Complete sparse matrix addition in C
Lesson 7	Sparse matrix using C++ classes
Lesson 8	Sparse matrix operator overloading in C++
Lesson 9	Polynomial representation
Lesson 10	Polynomial evaluation
Lesson 11	Polynomial addition algorithm

Lesson 9 taught how to store a polynomial as an array of terms.

Lesson 10 taught how to evaluate a polynomial for a given value of x .

Lesson 11 now teaches how to add two stored polynomials.

The important idea is that polynomial addition uses the same kind of merge logic that you already used in sparse matrix addition.

2. Main Goal of Lesson 11

The goal of Lesson 11 is to add two polynomials that are stored as arrays of terms.

Example:

$$P1 = 5x^4 + 2x^2 + 5$$

$$P2 = 6x^4 + 5x^3 + 9x^2 + 2x + 3$$

The result should be:

$$P1 + P2 = 11x^4 + 5x^3 + 11x^2 + 2x + 8$$

To do this, we compare the exponents of the terms.

Main rule:

Same exponent → add the coefficients
 Different exponent → copy the term with the larger exponent

3. Quick Reminder: What Is a Polynomial?

A polynomial is an expression made of terms.

Example:

$$3x^5 + 2x^4 + 5x^2 + 2x + 7$$

This polynomial has 5 terms:

$$3x^5$$

$$2x^4$$

$$5x^2$$

$$2x$$

$$7$$

Each term has two main parts:

Part	Meaning	Example in $3x^5$
Coefficient	Number multiplied by x	3
Exponent	Power of x	5

So:

$3x^5$

is stored as:

```
coefficient = 3  
exponent = 5
```

4. Polynomial Storage Structure

In this lesson, one term is stored using this structure:

```
struct term {  
    int coef;  
    int exp;  
};
```

Meaning:

Field	Meaning
coef	coefficient
exp	exponent

Example:

```
term t;  
t.coef = 5;  
t.exp = 4;
```

This represents:

$5x^4$

One full polynomial is stored using this structure:

```
struct poly {  
    int n;
```

```
struct term *t;  
};
```

Meaning:

Field	Meaning
n	number of terms in the polynomial
t	pointer to a dynamic array of terms

Important:

n = number of terms

It does not mean:

number of columns
highest exponent

Example:

$5x^4 + 2x^2 + 5$

This polynomial has 3 terms, so:

n = 3

Even though the highest exponent is 4, n is not 4.

5. How One Polynomial Is Stored

Take this polynomial:

$5x^4 + 2x^2 + 5$

It is stored as:

Index	Coefficient	Exponent	Term
0	5	4	$5x^4$
1	2	2	$2x^2$
2	5	0	5

In memory:

```
t[0] = {5, 4}
t[1] = {2, 2}
t[2] = {5, 0}
```

Remember:

$$5 = 5x^0$$

because:

$$x^0 = 1$$

So the constant term has exponent 0.

6. What Polynomial Addition Means

To add polynomials, we combine like terms.

Like terms are terms with the same exponent.

Example:

$$5x^4 \text{ and } 6x^4$$

Both terms have exponent 4, so they can be added:

$$5x^4 + 6x^4 = 11x^4$$

Only the coefficients are added:

$$5 + 6 = 11$$

The exponent stays the same:

$$4$$

Another example:

$$2x^2 + 9x^2 = 11x^2$$

Again:

$$2 + 9 = 11$$

The exponent stays .

Terms with different exponents cannot be combined.

Example:

$$5x^4 + 2x^3$$

These are not like terms.

So we do not write:

$$7x^?$$

Correct result:

$$5x^4 + 2x^3$$

7. Main Rule of Polynomial Addition

The main rule is:

If exponents are equal:
Add the coefficients.
Store one term.

If exponents are different:
Copy the term with the larger exponent first.

Why copy the larger exponent first?

Because we store the result in descending exponent order.

Descending order means highest exponent to lowest exponent.

Example:

x^5, x^4, x^2, x^1, x^0

This order makes the polynomial easy to read and easy to process.

8. Required Order: Descending Exponent Order

For this algorithm to work correctly, both polynomials should be sorted by exponent from largest to smallest.

Good order:

$5x^4 + 2x^2 + 5x^0$

Exponents:

4, 2, 0

Bad order:

$2x^2 + 5x^4 + 5x^0$

Exponents:

2, 4, 0

Why bad order is a problem:

The algorithm moves forward only.

It does not go back and search previous terms again.

So if the terms are not sorted, the result may be wrong.

This is the same idea as sparse matrix addition:

- Sparse matrix addition needs elements sorted by row and column.
- Polynomial addition needs terms sorted by exponent.

9. Example Polynomials for Lesson 11

We will use this example:

$$P1 = 5x^4 + 2x^2 + 5$$

$$P2 = 6x^4 + 5x^3 + 9x^2 + 2x + 3$$

Stored form of P1:

Index	Coefficient	Exponent	Term
0	5	4	$5x^4$
1	2	2	$2x^2$
2	5	0	$5x^0$

Stored form of P2:

Index	Coefficient	Exponent	Term
0	6	4	$6x^4$
1	5	3	$5x^3$
2	9	2	$9x^2$
3	2	1	$2x^1$
4	3	0	$3x^0$

The result should combine terms with the same exponent.

10. Result Size

When adding two polynomials, we do not know the final number of terms at the beginning.

Suppose:

$$\begin{aligned} P1.n &= 3 \\ P2.n &= 5 \end{aligned}$$

The maximum possible number of result terms is:

$$P1.n + P2.n = 3 + 5 = 8$$

This worst case happens when no exponents match.

Example worst case:

$$\begin{aligned} P1 &= 5x^4 + 2x^2 \\ P2 &= 7x^3 + 9x^1 \end{aligned}$$

No exponents are the same.

So all terms are copied into the result.

But the actual result can be smaller if some exponents match.

Example:

$$5x^4 + 6x^4 = 11x^4$$

Two input terms become one output term.

So we first allocate the maximum possible space:

$$P1.n + P2.n$$

Then, at the end, we store the actual number of terms.

The actual count is stored in `k`.

So we write:

```
sum->n = k;
```

Do not write:

```
sum->n = p1->n + p2->n;
```

That would store the maximum capacity, not the actual number of result terms.

11. The Three Index Variables: `i`, `j`, and `k`

Polynomial addition uses three index variables:

```
i = current term in P1  
j = current term in P2  
k = current position in the result polynomial
```

At the beginning:

```
i = 0;  
j = 0;  
k = 0;
```

Meaning:

```
i points to P1.t[0]  
j points to P2.t[0]  
k points to result.t[0]
```

Simple mental model:

```
i is a finger pointing at the current term in P1.  
j is a finger pointing at the current term in P2.  
k is a finger pointing at the next empty place in the result.
```

After copying or adding a term, the correct pointer moves forward.

12. Core Merge Idea

Polynomial addition is a merge operation.

At each step, compare:

```
P1.t[i].exp  
P2.t[j].exp
```

There are only three possible cases:

1. **P1** exponent is larger.
2. **P2** exponent is larger.
3. Both exponents are equal.

Each case has a different action.

13. Merge Rule 1: **P1** Exponent Is Larger

Example:

```
P1 term = 5x^4  
P2 term = 5x^3
```

Compare exponents:

```
4 > 3
```

So the **P1** term should come first in the result.

Action:

```
Copy P1 term into the result.  
Move i forward.  
Move k forward.
```

Code pattern:

```
sum->t[k++] = p1->t[i++];
```

Meaning:

```
Copy p1->t[i] into sum->t[k]
Then increase i
Then increase k
```

14. Merge Rule 2: P2 Exponent Is Larger

Example:

```
P1 term = 2x^2
P2 term = 5x^3
```

Compare exponents:

```
2 < 3
```

So the P2 term should come first in the result.

Action:

```
Copy P2 term into the result.
Move j forward.
Move k forward.
```

Code pattern:

```
sum->t[k++] = p2->t[j++];
```

Meaning:

```
Copy p2->t[j] into sum->t[k]
Then increase j
Then increase k
```

15. Merge Rule 3: Exponents Are Equal

Example:

```
P1 term = 5x^4  
P2 term = 6x^4
```

Compare exponents:

```
4 == 4
```

The exponents are the same, so the terms can be added.

Action:

```
Keep the same exponent.  
Add the coefficients.  
Store one result term.  
Move i forward.  
Move j forward.  
Move k forward.
```

Code pattern:

```
sum->t[k].exp = p1->t[i].exp;  
sum->t[k].coef = p1->t[i].coef + p2->t[j].coef;  
i++;  
j++;  
k++;
```

Example result:

```
5x^4 + 6x^4 = 11x^4
```

The exponent stays `4`.

The coefficient becomes:

$$5 + 6 = 11$$

16. Merge Rules Summary Table

Condition	Action	Pointer movement
<code>P1.exp > P2.exp</code>	Copy <code>P1</code> term	<code>i++</code> , <code>k++</code>
<code>P1.exp < P2.exp</code>	Copy <code>P2</code> term	<code>j++</code> , <code>k++</code>
<code>P1.exp == P2.exp</code>	Add coefficients and store once	<code>i++</code> , <code>j++</code> , <code>k++</code>
One polynomial ends	Copy leftovers from the other polynomial	Move source pointer and <code>k</code>

17. Full Dry Run

Let:

$$P1 = 5x^4 + 2x^2 + 5$$

$$P2 = 6x^4 + 5x^3 + 9x^2 + 2x + 3$$

Stored form:

P1:

Index	Coef	Exp
0	5	4
1	2	2
2	5	0

P2:

Index	Coef	Exp
0	6	4
1	5	3
2	9	2
3	2	1
4	3	0

Initial values:

```
i = 0  
j = 0  
k = 0
```

Step 1

Current terms:

```
P1[i] = P1[0] = 5x^4  
P2[j] = P2[0] = 6x^4
```

Compare exponents:

```
4 == 4
```

The exponents are equal.

So add the coefficients:

```
5 + 6 = 11
```

Store:

```
P3[0] = 11x^4
```

Move:

```
i = 1  
j = 1  
k = 1
```

Current result:

```
11x^4
```

Step 2

Current terms:

$$\begin{aligned}P1[i] &= P1[1] = 2x^2 \\ P2[j] &= P2[1] = 5x^3\end{aligned}$$

Compare exponents:

$$2 < 3$$

P2 has the larger exponent.

So copy the P2 term:

$$P3[1] = 5x^3$$

Move:

$$\begin{aligned}j &= 2 \\ k &= 2\end{aligned}$$

i stays the same:

$$i = 1$$

Current result:

$$11x^4 + 5x^3$$

Step 3

Current terms:

$$\begin{aligned}P1[i] &= P1[1] = 2x^2 \\ P2[j] &= P2[2] = 9x^2\end{aligned}$$

Compare exponents:

$$2 == 2$$

The exponents are equal.

So add the coefficients:

$$2 + 9 = 11$$

Store:

$$P3[2] = 11x^2$$

Move:

$$\begin{aligned} i &= 2 \\ j &= 3 \\ k &= 3 \end{aligned}$$

Current result:

$$11x^4 + 5x^3 + 11x^2$$

Step 4

Current terms:

$$\begin{aligned} P1[i] &= P1[2] = 5x^0 \\ P2[j] &= P2[3] = 2x^1 \end{aligned}$$

Compare exponents:

$$0 < 1$$

P2 has the larger exponent.

So copy the **P2** term:

$$P3[3] = 2x^1$$

Move:

$$\begin{aligned} j &= 4 \\ k &= 4 \end{aligned}$$

i stays the same:

$$i = 2$$

Current result:

$$11x^4 + 5x^3 + 11x^2 + 2x^1$$

Step 5

Current terms:

$$\begin{aligned} P1[i] &= P1[2] = 5x^0 \\ P2[j] &= P2[4] = 3x^0 \end{aligned}$$

Compare exponents:

$$0 == 0$$

The exponents are equal.

So add the coefficients:

$$5 + 3 = 8$$

Store:

$$P3[4] = 8x^0$$

Move:

```
i = 3  
j = 5  
k = 5
```

Now both lists are finished.

Final result:

$$P3 = 11x^4 + 5x^3 + 11x^2 + 2x^1 + 8x^0$$

Usually we write this as:

$$P3 = 11x^4 + 5x^3 + 11x^2 + 2x + 8$$

18. Why We Copy Leftovers

The main loop works only while both polynomials still have terms.

Loop condition:

```
while (i < p1->n && j < p2->n)
```

This means:

```
Continue only while P1 still has terms and P2 still has terms.
```

If one polynomial finishes first, the other polynomial may still have remaining terms.

Those remaining terms are called leftovers.

They must be copied into the result.

Why?

Because if a term exists only in one polynomial, it stays unchanged in the sum.

Example:

```
P1 = 7x^5 + 3x^2  
P2 = 4x^2
```

After adding $3x^2 + 4x^2$, the term $7x^5$ still exists.

It must be copied into the result.

19. Copying Leftovers from P1

If P2 finishes first, copy the remaining terms from P1.

Code:

```
while (i < p1->n) {  
    sum->t[k++] = p1->t[i++];  
}
```

Meaning:

```
While P1 still has terms:  
    Copy the current P1 term into the result.  
    Move i forward.  
    Move k forward.
```

20. Copying Leftovers from P2

If P1 finishes first, copy the remaining terms from P2.

Code:

```
while (j < p2->n) {  
    sum->t[k++] = p2->t[j++];  
}
```

Meaning:

```
While P2 still has terms:  
    Copy the current P2 term into the result.  
    Move j forward.  
    Move k forward.
```

21. Beginner-Friendly Algorithm

```
Create result array with size P1.n + P2.n  
  
Set i = 0  
Set j = 0  
Set k = 0  
  
While i is inside P1 and j is inside P2:  
  
    If P1 exponent is greater than P2 exponent:  
        Copy P1 term to result  
        Move i  
        Move k  
  
    Else if P1 exponent is less than P2 exponent:  
        Copy P2 term to result  
        Move j  
        Move k  
  
    Else:  
        Exponents are equal  
        Add the coefficients  
        Store one result term  
        Move i  
        Move j  
        Move k  
  
Copy any leftover terms from P1  
Copy any leftover terms from P2  
  
Set result.n = k  
Return result
```

22. C++-Style Code Pattern

```
struct poly* add(struct poly *p1, struct poly *p2) {
    int i = 0;
    int j = 0;
    int k = 0;

    struct poly *sum = new poly;
    sum->t = new term[p1->n + p2->n];

    while (i < p1->n && j < p2->n) {
        if (p1->t[i].exp > p2->t[j].exp) {
            sum->t[k++] = p1->t[i++];
        }
        else if (p1->t[i].exp < p2->t[j].exp) {
            sum->t[k++] = p2->t[j++];
        }
        else {
            sum->t[k].exp = p1->t[i].exp;
            sum->t[k].coef = p1->t[i].coef + p2->t[j].coef;
            i++;
            j++;
            k++;
        }
    }

    while (i < p1->n) {
        sum->t[k++] = p1->t[i++];
    }

    while (j < p2->n) {
        sum->t[k++] = p2->t[j++];
    }

    sum->n = k;
    return sum;
}
```

23. Explanation of the Code

Function header

```
struct poly* add(struct poly *p1, struct poly *p2)
```

Meaning:

Part	Meaning
<code>struct poly*</code>	The function returns a pointer to a polynomial
<code>add</code>	Function name
<code>struct poly *p1</code>	Pointer to the first polynomial
<code>struct poly *p2</code>	Pointer to the second polynomial

The function returns a pointer because the result polynomial is created inside the function.

Initial pointers

```
int i = 0;  
int j = 0;  
int k = 0;
```

Meaning:

i starts at the first term of p1.
j starts at the first term of p2.
k starts at the first empty position of sum.

Creating the result polynomial

```
struct poly *sum = new poly;  
sum->t = new term[p1->n + p2->n];
```

This creates:

1. A new polynomial structure.
2. A dynamic array large enough for the worst-case result.

Worst-case size:

$p1 \rightarrow n + p2 \rightarrow n$

Main loop

```
while (i < p1->n && j < p2->n)
```

This loop continues while both polynomials still have terms to compare.

If **p1** exponent is larger

```
if (p1->t[i].exp > p2->t[j].exp) {  
    sum->t[k++] = p1->t[i++];  
}
```

This copies the **p1** term into the result.

If **p2** exponent is larger

```
else if (p1->t[i].exp < p2->t[j].exp) {  
    sum->t[k++] = p2->t[j++];  
}
```

This copies the **p2** term into the result.

If exponents are equal

```
else {  
    sum->t[k].exp = p1->t[i].exp;  
    sum->t[k].coef = p1->t[i].coef + p2->t[j].coef;  
    i++;  
    j++;  
    k++;  
}
```

This adds the coefficients and stores one combined term.

Copy leftovers

```
while (i < p1->n) {  
    sum->t[k++] = p1->t[i++];  
}  
  
while (j < p2->n) {  
    sum->t[k++] = p2->t[j++];  
}
```

These loops copy any terms that remain after one polynomial ends.

Set the final term count

```
sum->n = k;
```

This stores the actual number of result terms.

This is important because the allocated array may be larger than the number of terms actually used.

24. Structure Assignment

In this line:

```
sum->t[k++] = p1->t[i++];
```

one whole `term` structure is copied.

That means both fields are copied:

```
coefficient  
exponent
```

Example:

```
p1->t[i] = {5, 4}
```

After copying:

```
sum->t[k] = {5, 4}
```

So structure assignment saves us from writing:

```
sum->t[k].coef = p1->t[i].coef;  
sum->t[k].exp = p1->t[i].exp;
```

Both versions work, but structure assignment is shorter.

25. Important Note About Zero Coefficients

Sometimes two terms cancel each other.

Example:

```
5x^2 + -5x^2 = 0x^2
```

Mathematically, this term should disappear from the result.

A simple beginner version may still store:

```
0x^2
```

But a cleaner implementation should skip terms whose coefficient becomes zero.

Cleaner idea:

```
int c = p1->t[i].coef + p2->t[j].coef;  
if (c != 0) {  
    sum->t[k].coef = c;  
    sum->t[k].exp = p1->t[i].exp;  
    k++;  
}  
i++;  
j++;
```

For Lesson 11, first understand the basic merge logic. Then add zero-skipping as an improvement.

26. Complexity Analysis

Let:

P1 has n terms
P2 has m terms

Each term is visited once.

So the time complexity is:

$O(n + m)$

Why?

Because the algorithm does not compare every term with every other term.

It simply walks through both arrays once.

The result array may need to store all terms in the worst case.

So the space complexity is:

$O(n + m)$

27. Why the Algorithm Is Efficient

A slower method would be:

For every term in P1:
 Search every term in P2 for the same exponent

That could take:

$O(n \times m)$

But Lesson 11 uses sorted arrays and a merge method.

That gives:

$$O(n + m)$$

This is faster because each term is processed only once.

28. Common Beginner Mistakes

Mistake 1: Adding different exponents

Wrong:

$$5x^4 + 2x^3 = 7x^7$$

Correct:

$$5x^4 + 2x^3$$

Only terms with the same exponent can be combined.

Mistake 2: Changing the exponent when adding

Wrong:

$$5x^4 + 6x^4 = 11x^8$$

Correct:

$$5x^4 + 6x^4 = 11x^4$$

The coefficients are added.

The exponent stays the same.

Mistake 3: Setting result size incorrectly

Wrong:

```
sum->n = p1->n + p2->n;
```

Correct:

```
sum->n = k;
```

`k` is the actual number of terms written.

Mistake 4: Forgetting leftovers

If one polynomial ends first, the remaining terms in the other polynomial must still be copied.

Mistake 5: Confusing the meaning of `n`

In polynomial code:

```
n = number of terms
```

It is not:

```
number of columns  
highest exponent
```

Mistake 6: Giving input in random exponent order

The merge algorithm assumes descending exponent order.

Good:

```
7x^5 + 4x^3 + 2x^1
```

Bad:

```
4x^3 + 7x^5 + 2x^1
```

29. Quick Practice Example

Add:

$$\begin{aligned} P1 &= 4x^5 + 3x^2 + 7 \\ P2 &= 2x^4 + 6x^2 + 1 \end{aligned}$$

Step-by-step:

$4x^5$ has no match, so copy it.
 $2x^4$ has no match, so copy it.
 $3x^2$ and $6x^2$ have the same exponent, so add them.
 7 and 1 are constants, so add them.

Calculations:

$$\begin{aligned} 3x^2 + 6x^2 &= 9x^2 \\ 7 + 1 &= 8 \end{aligned}$$

Final answer:

$$4x^5 + 2x^4 + 9x^2 + 8$$

30. Lesson 11 Summary

Polynomial addition is based on exponent comparison.

The polynomial is stored as an array of terms.

Each term stores:

coefficient
exponent

The full polynomial stores:

n = number of terms
t = dynamic term array

To add two polynomials:

1. Keep both term arrays sorted by descending exponent.
2. Use three indexes: `i`, `j`, and `k`.
3. Compare exponents.
4. If one exponent is larger, copy that term.
5. If exponents are equal, add coefficients and store one term.
6. Copy leftovers after one polynomial ends.
7. Set the final result count using `k`.

Final complexity:

Time: $O(n + m)$
Space: $O(n + m)$

The main mental model:

Polynomial addition is merging two sorted term lists.

This is the same style of thinking used earlier in sparse matrix addition, but now the comparison key is exponent instead of row and column.